

The Python Application

ipecho - a FastAPI service that returns the client's originating IP, geolocated

Carlos Pereyra

July 20, 2026

ipecho is a small FastAPI application whose job is to return the originating IP address of the client that called it, enriched with geolocation, and to expose that data for a map dashboard. Despite its size, it demonstrates several real production concerns: getting the true client IP behind a proxy, optional geo-IP enrichment, in-memory aggregation, Prometheus metrics, and build metadata.

FIELD	VALUE
Language	Python 3.12
Framework	FastAPI + Uvicorn
Source	app/main.py
Tests	app/test_main.py (pytest)
Geo database	MaxMind GeoLite2-City (optional, via GEOIP_DB)
Metrics	Prometheus (prometheus-client)

1. Endpoints

The service exposes eight endpoints. The root and /myip endpoints record the request into the in-memory store that feeds the dashboard.

METHOD / PATH	RETURNS
GET /	JSON: ip, country, city, lat, lon, pod hostname
GET /myip	The raw client IP as plain text
GET /version	Build info: version, git_sha, build_time
GET /health	Liveness probe payload
GET /ready	Readiness probe payload
GET /stats	Aggregated geo points (feeds the map)
GET /log	Last 100 seen IPs (ip, country, city, lat, lon, ts)
GET /metrics	Prometheus metrics

2. Resolving the real client IP

The most interesting problem is getting the true client IP when the app sits behind an ingress and a CDN. The connection the app sees is the last hop (the proxy), not the client. Headers carry the real address, but they must be trusted in the right order.

Why the order matters

Behind Traefik, a client-supplied X-Forwarded-For is overwritten by the proxy with the internal hop (for example 10.42.x.x) - Traefik does not trust arbitrary inbound XFF, which is correct (otherwise anyone could spoof their IP). When the app is fronted by Cloudflare, the real client IP arrives in the CF-Connecting-IP header, which the proxy forwards untouched. Therefore CF-Connecting-IP must be preferred over X-Forwarded-For.

Implementation

```
TRUSTED_IP_HEADERS = ("cf-connecting-ip", "true-client-ip")

def client_ip(request: Request) → str:
    for header in TRUSTED_IP_HEADERS:
        value = request.headers.get(header)
        if value:
            return value.strip()
    forwarded = request.headers.get("x-forwarded-for")
    if forwarded:
        return forwarded.split(",")[0].strip()
    real = request.headers.get("x-real-ip")
    if real:
        return real.strip()
    if request.client:
        return request.client.host
    return "unknown"
```

Observed in production through the tunnel: the JSON response showed ip = the real public IP while forwarded_for = 10.42.2.1 (the internal Traefik hop) - proof the precedence works.

3. Geolocation

If a GeoLite2-City database is available (path in the GEOIP_DB environment variable), the app resolves country, city and latitude/longitude from the IP. If it is not available, it falls back to the country from Cloudflare's CF-IPCountry header. This makes the app degrade gracefully: tests and local runs work with no database, and the deployed pod gets a full database mounted by an init container.

```
def geolocate(ip: str, request: Request):
    country = request.headers.get("cf-ipcountry", "").upper()
    city = None; lat = None; lon = None
    if _geoip_reader:
        try:
            r = _geoip_reader.city(ip)
            country = (r.country.iso_code or country or "").upper()
            city = r.city.name
            lat = r.location.latitude
            lon = r.location.longitude
        except Exception:
            pass
    if not country or country in ("XX", "T1"):
        country = "ZZ"
    return country, city, lat, lon
```

Anycast addresses such as 1.1.1.1 have no location in the database and correctly resolve to ZZ / null.

4. In-memory store and aggregation

Seen IPs are aggregated by rounded coordinates into a points dictionary (for the map) and kept in a bounded deque of the last 100 requests (for the log). A threading lock guards both, since Uvicorn can process requests concurrently.

```
def record(ip, country, city, lat, lon):
    ts = int(time.time())
    with _lock:
        _country_counts[country] += 1
        _recent.appendleft({"ip": ip, "country": country, "city": city,
                           "lat": lat, "lon": lon, "ts": ts})
    if lat is not None and lon is not None:
        key = (round(lat, 2), round(lon, 2))
        loc = _locations.get(key)
        if loc:
            loc["count"] += 1
        else:
            _locations[key] = {"lat": lat, "lon": lon, "country": country,
                               "city": city, "count": 1}
    requests_total.labels(country=country).inc()
```

The scaling trade-off

Because this state lives in the memory of a single pod, /stats and /log are per-replica. On Kubernetes the app therefore runs with a single replica so the map is consistent. Scaling horizontally would require a shared store (for example Redis or Postgres) to make the app stateless. This is a deliberate, documented trade-off and a good interview talking point: "your app holds state in memory - how do you scale it?"

5. Prometheus metrics and build info

The app exposes a Prometheus counter labelled by country, and a /version endpoint whose values are baked into the image at build time via Docker build arguments (GIT_SHA, BUILD_TIME, APP_VERSION). The dashboard reads /version to show exactly which commit is running.

```
requests_total = PromCounter("ipecho_requests_total", "Total requests seen", ["country"])

BUILD_INFO = {
    "version": os.getenv("APP_VERSION", "0.4.0"),
    "git_sha": os.getenv("GIT_SHA", "dev"),
    "build_time": os.getenv("BUILD_TIME", "unknown"),
}
```

6. Testing

The pytest suite covers the IP precedence rules, the geo fallback, the plain-text and JSON responses, the aggregation, and the metrics. Tests reset the in-memory store between cases and run without any geo database, exercising the graceful-fallback path.

```
def test_root_prefers_cf_connecting_ip_over_forwarded_for():
    resp = client.get("/", headers={"CF-Connecting-IP": "198.51.100.9",
                                    "X-Forwarded-For": "10.42.0.0"})
    assert resp.json()["ip"] == "198.51.100.9"

def test_myip_returns_plain_ip():
    resp = client.get("/myip", headers={"CF-Connecting-IP": "203.0.113.5"})
    assert resp.text.strip() == "203.0.113.5"
```

7. Dependencies

Runtime dependencies are intentionally minimal.

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
prometheus-client==0.21.1
geoip2==4.8.0
```

Design summary: prefer trusted proxy headers for the client IP, enrich with an optional geo database that degrades gracefully, aggregate in memory behind a lock, expose Prometheus metrics and build metadata, and keep the dependency surface small.